```
!pip -q install nltk scikit-learn

import pandas as pd
import re
import nltk

from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

nltk.download("punkt")
nltk.download("punkt_tab")
nltk.download("stopwords")
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
True
```

```
from google.colab import files

uploaded = files.upload()
df = pd.read_csv(next(iter(uploaded)), encoding='latin-1')

print("First five records:")
display(df.head())

print("Shape:", df.shape)
print("Columns:", df.columns.tolist())
```

Choose Files customer review.csv

customer review.csv(text/csv) - 67546 bytes, last modified: 10/12/2026 - 100% done

```
Saving customer review.csv to customer review (2).csv
First five records:
```

	Review Title	Customer name	Rating	Date	Category	Comments	Useful
0	Another Midrange killer Smartphone by Xiaomi	Rishikumar Thakur	4.0 out of 5 stars	on 1 October 2018	Display	Another Midrange killer Smartphone by Xiaomi\n...	
1	vry small size mobile	Raza ji	3.0 out of 5 stars	on 15 September 2018	Others	All ok but vry small size mobile	7 people found this helpful
2	Full display not working in all application.	Vaibhav Patel	3.0 out of 5 stars	on 18 September 2018	Others	Quite good	7 people found this helpful
3	Value for Money	Amazon Customer	5.0 out of 5 stars	on 28 September 2018	Display	Redmi has always have been the the king of bud...	2 people found this helpful
4	Not worth for the money	Sudhakaran Wadakkancheri	2.0 out of 5 stars	on 18 September 2018	Others	worst product from MI. I am a hardcore fan of ...	6 people found this helpful

```
df = df[["Comments"]].rename(columns={"Comments": "Text"}).dropna().drop_duplicates()
df = df.head(10000).reset_index(drop=True)
```

```
stop_words = set(stopwords.words("english"))

def preprocess_text(text):
    text = str(text).lower()
    text = re.sub(r"[^a-z\s]", " ", text)

    tokens = word_tokenize(text)

    tokens = [
        word for word in tokens
        if word not in stop_words and len(word) > 1
    ]

    return " ".join(tokens)

df["Cleaned_Text"] = df["Text"].apply(preprocess_text)

# Initialize and fit TF-IDF vectorizer so the next cells can run
vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(df["Cleaned_Text"])

display(df.head())
```

	Text	Cleaned_Text
0	Another Midrange killer Smartphone by Xiaomi\n...	another midrange killer smartphone xiaomi majo...
1	All ok but vry small size mobile	ok vry small size mobile
2	Quite good	quite good
3	Redmi has always have been the the king of bud...	redmi always king budget segment yet another g...
4	worst product from MI. I am a hardcore fan of ...	worst product mi hardcore fan mi one really di...

```
vectorizer = TfidfVectorizer()

tfidf_matrix = vectorizer.fit_transform(df["Cleaned_Text"])

print("TF-IDF matrix shape:", tfidf_matrix.shape)
print("Vocabulary size:", len(vectorizer.vocabulary_))
```

```
TF-IDF matrix shape: (216, 1039)
Vocabulary size: 1039
```

```
def search_reviews(query, top_n=5):
    query_cleaned = preprocess_text(query)

    query_vector = vectorizer.transform([query_cleaned])

    scores = cosine_similarity(
        query_vector,
        tfidf_matrix
    ).flatten()

    indices = scores.argsort()[::-1][:top_n]

    results = pd.DataFrame({
        "Similarity Score": scores[indices],
        "Review": df.iloc[indices]["Text"].values
    })

    return results

query = "good taste and excellent quality"

display(search_reviews(query, top_n=5))
```

	Similarity Score	Review
0	0.791670	Excellent mobile good display quality
1	0.782458	Excellent
2	0.622704	Good quality
3	0.366318	good
4	0.366318	Too good

```
queries = [
    "good product quality",
    "bad packaging",
    "excellent taste",
    "fast delivery"
]

for query in queries:
    print("\nSearch query:", query)
    display(search_reviews(query, top_n=3))
```

```
Search query: good product quality
```

	Similarity Score	Review
0	0.783991	Good product
1	0.783991	Very good product
2	0.783991	Good Product

```
Search query: bad packaging
```

	Similarity Score	Review